

String Class API

```
public class String {  
    /** Returns the length of this string. */  
    public int length() { ... }  
  
    /** Returns the character at the specified index (position) */  
    public char charAt(int index) { ... }  
  
    /** Returns a new string that is a substring of this string starting at the beginIndex  
     * For example: "asdfjkl".substring(4) evaluates to "jkl" */  
    public String substring(int beginIndex) { ... }  
}
```

Random Class API

```
public class Random {  
    /** Creates a new random number generator. */  
    public Random() { ... }  
  
    /** Creates a new random number generator using a seed. If two instances of Random are created  
     * with the same seed, they will generate and return identical sequences of numbers. Runs in  
     *  $\Theta(1)$  time. */  
    public Random(long seed) { ... }  
  
    /** Returns the next pseudorandom int from this random number generator's sequence. Runs in  
     *  $\Theta(1)$  time. */  
    public int nextInt() { ... }  
  
    /** Returns a pseudorandom int value between 0 (inclusive) and the specified value (exclusive),  
     * drawn from this random number generator's sequence. Runs in  $\Theta(1)$  time. */  
    public int nextInt(int bound) { ... }  
}
```

MinHeap Class API

```
public class MinHeap<E> {  
    /** Create a MinHeap<E>. Runs in  $\Theta(1)$  time. */  
    public MinHeap() { ... }  
  
    /** Inserts an element into the heap. Runs in  $\Theta(\log(N))$  time. */  
    public void insert(E element) { ... }  
  
    /** Deletes and returns the minimum element in the heap. If empty, return null.  
     * Runs in  $\Theta(\log(N))$  time. */  
    public E deleteMin() { ... }  
  
    /** Determines whether the heap is empty or not. Runs in  $\Theta(1)$  time. */  
    public boolean isEmpty() { ... }  
}
```

```

    /** Returns the size of the heap. Runs in  $\Theta(1)$  time. */
    public int size() { ... }

    /** Returns, but does not delete, the minimum element in the heap. This method is
     * non-destructive. Runs in  $\Theta(1)$  time. */
    public E findMin() { ... }
}

```

Graph Class API

```

public class Graph {
    /** Creates a new graph with V nodes (numbered from 0 to V-1). Runs in  $\Theta(V)$  time. */
    public Graph(int V) { ... }

    /** Creates a deep copy of graph g. Runs in  $\Theta(|E|)$  time */
    public Graph(Graph g) { ... }

    /** Adds an edge to the graph. If an edge already exists between g.from and g.to, it is replaced.
     Runs in  $\Theta(1)$  time. */
    public void addEdge(GraphEdge g) { ... }

    /** Returns a set of all edges leaving the given node. Runs in  $\Theta(|V|)$  time. */
    public Set<GraphEdge> neighbors(int from) { ... }

    /** Returns the number of vertices. Runs in  $\Theta(1)$  time. */
    public int size() { ... }
}

```

GraphEdge Class API

```

public class GraphEdge implements Comparable {
    public int from;
    public int to;
    public int weight;
    /** Creates a directed edge of the given weight between the given nodes. Runs in  $\Theta(1)$  time. */
    public GraphEdge(int from, int to, int weight) { ... }

    /** Compares two edges according to their weights. For example, an edge with weight 4 will be
     less than an edge with weight 9. Runs in  $\Theta(1)$  time. */
    public int compareTo(GraphEdge e) { ... }
}

```

Queue Class API

```

public class Queue<E> {
    /** Create a queue*/
    public Queue() { ... }
}

```

```

    /** Create a queue, then inserts all elements of lst one at a time into the queue. Runs in  $\Theta(N)$ 
    time.*/
    public Queue(List<E> lst) { ... }

    /** Inserts the specified element into this queue returning true on success. Runs in  $\Theta(1)$  time.*/
    public boolean add(E e) { ... }

    /** Retrieves, but does not remove, the element at the head of the queue, or returns null if this
    queue is empty. Runs in  $\Theta(1)$  time.*/
    public E peek() { ... }

    /** Retrieves and removes the element at the head of the queue. Runs in  $\Theta(1)$  time.*/
    public E remove() { ... }

    /** Returns the size of the queue. Runs in  $\Theta(1)$  time.*/
    public int size() { ... }

}

```

ArrayList Class API

```

public class ArrayList<E> implements List<E> {
    /** Appends the specified element to the end of this list. Runs in  $\Theta(1)$  time. */
    public boolean add(E e) { ... }

    /** Returns the element at the specified position in this list. */
    public E get(int index) { ... }

    /** Replaces the element at the specified position in this list with the specified element. */
    public E set(int index, E element) { ... }

    /** Returns the number of elements in this list. */
    public int size() { ... }
}

```

Math Class API

```

public class Math {
    /** Returns the value of the first argument raised to the power of the second argument. */
    public static double pow(double a, double b) { ... }

    /** Returns the smaller of two int values. */
    public static int min(int a, int b) { ... }

    /** Returns the greater of two int values. */
    public static int max(int a, int b) { ... }
}

```


